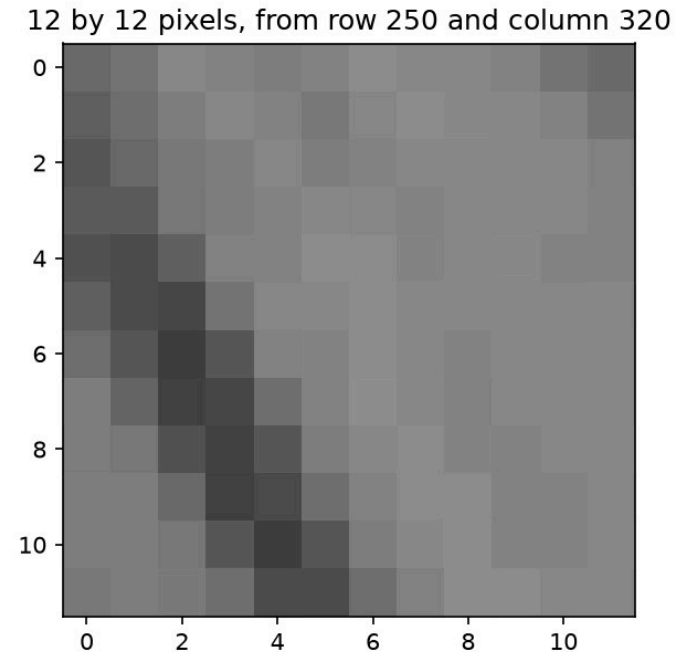
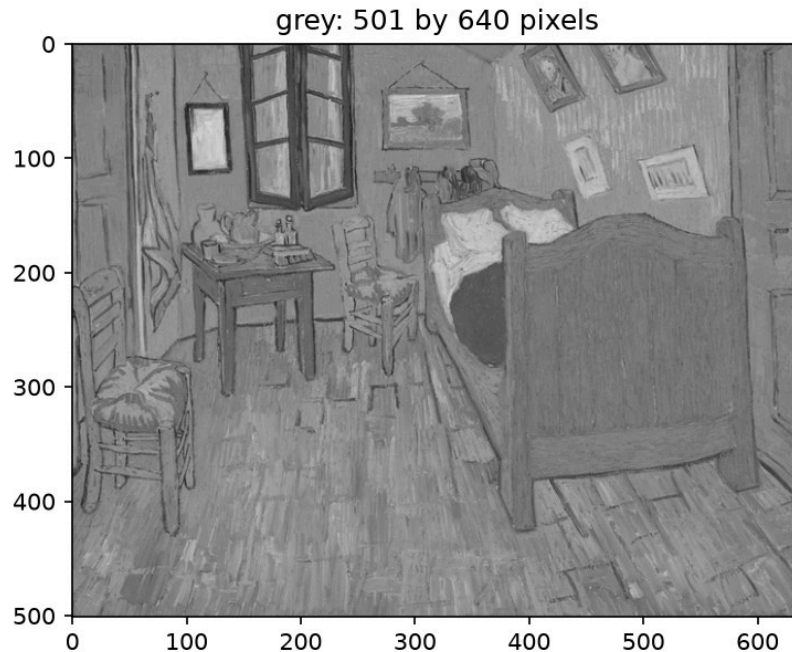


An Image Is an Array

A grey image as a matrix, a colour image as an array with three dimensions, and operations that touch every pixel at once.

A grey image is a matrix



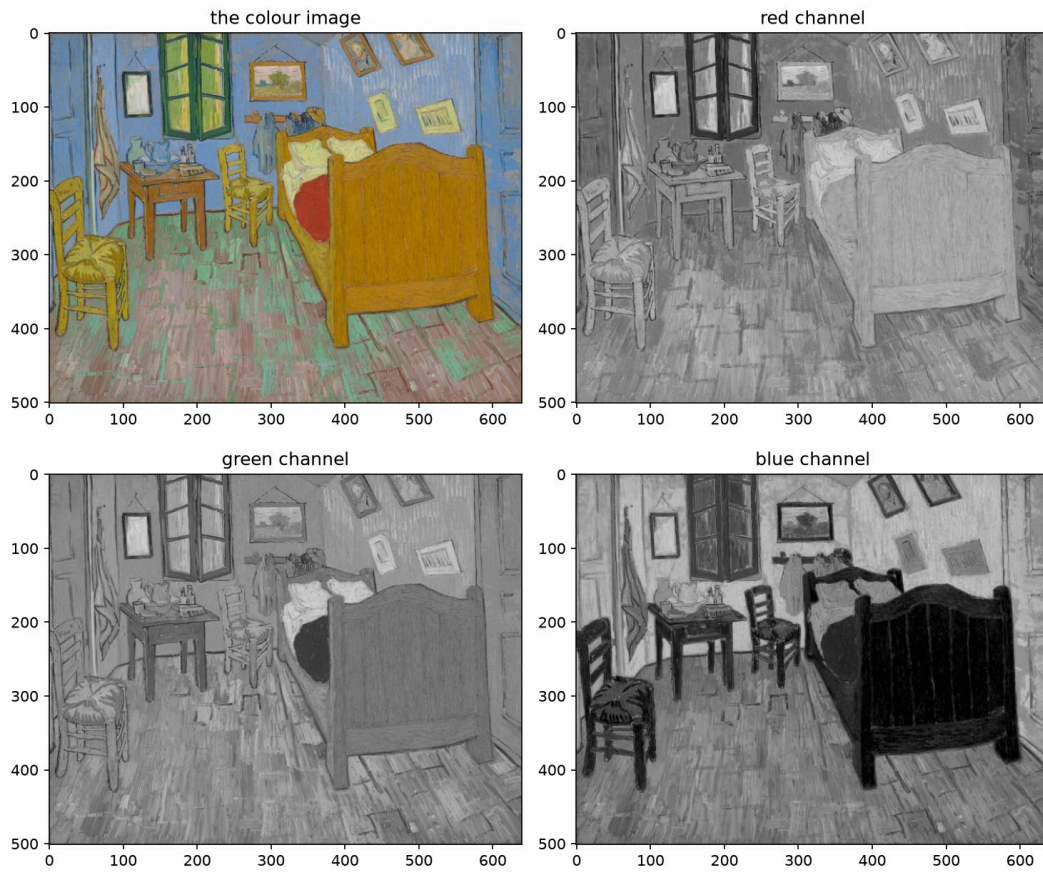
shape of grey: (501, 640), which is $H = 501$ rows and $W = 640$ columns of pixels, 320,640 numbers
darkest value 0.10, brightest value 0.85, mean 0.52

A **pixel** holds one number: 0 is black, 1 is white. An image with H rows and W columns of pixels is a matrix with H rows and W columns. The enlarged block is 12 by 12 entries of that matrix, `grey[r:r + 12, c:c + 12]`.

A colour image: three numbers for each pixel

shape of image: (501, 640, 3), which is 961,920 numbers

the pixel in row 0 and column 0: [0.55 0.47 0.37] (red, green, blue)



- `image[:, :, 0]` is the **red channel**: a matrix with H rows and W columns. A colon alone means all the positions along that dimension.
- An array with any number of dimensions is called a **tensor**. A matrix is a tensor with two dimensions.

From colour to grey

A common rule gives every pixel the brightness **grey** = **0.299** · **red** + **0.587** · **green** + **0.114** · **blue**. Green has the largest weight because the eye is most sensitive to green light.

```
def grey_loops(image):
    H, W = image.shape[0], image.shape[1]
    result = np.empty((H, W))
    for i in range(H):
        for j in range(W):
            result[i, j] = 0.299 * image[i, j, 0] + 0.587 * image[i, j, 1] + 0.114 * image[i, j, 2]
    return result

def grey_vectorized(image):
    return 0.299 * image[:, :, 0] + 0.587 * image[:, :, 1] + 0.114 * image[:, :, 2]
```

Two functions written by an AI assistant to the same requirements: one with two nested loops over the pixels, one with arithmetic on whole channels.

Loops against whole channels

Correct: both functions reproduce the grey image of Step 11.

Image 7: 501 by 640 pixels

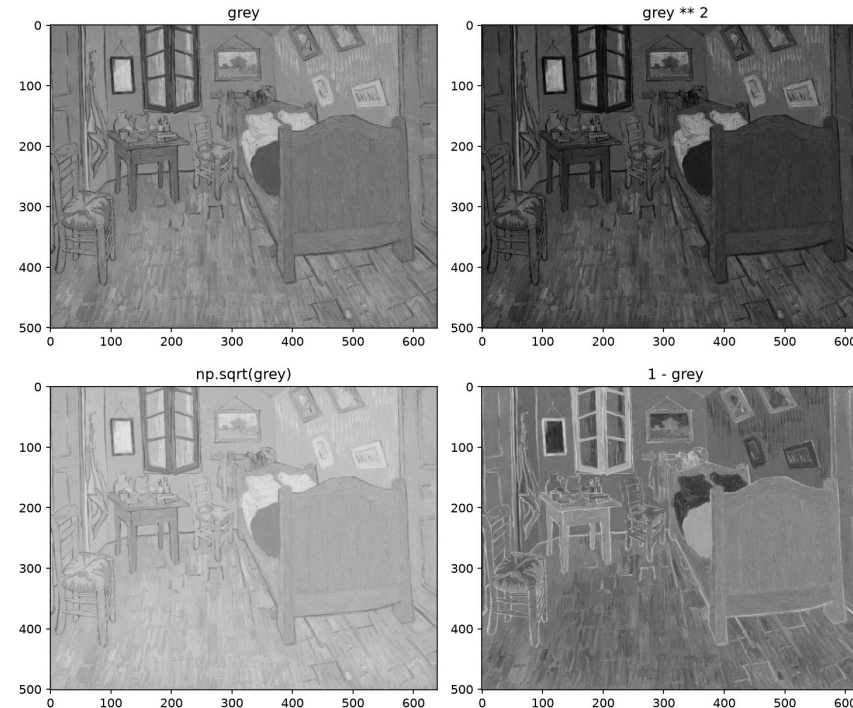
version	seconds per call	nanoseconds per pixel
loops	0.1364	425.4
vectorized	0.0027	8.4

The loops take 50 times as long as the vectorized version.

- Both functions reproduce the stored grey image. The time is the difference: about 400 nanoseconds per pixel with loops, about 8 with arithmetic on whole channels.
- The same factor as in Act 1, for the same reason: the loop runs in the interpreter, or it runs inside NumPy.

One function for every entry

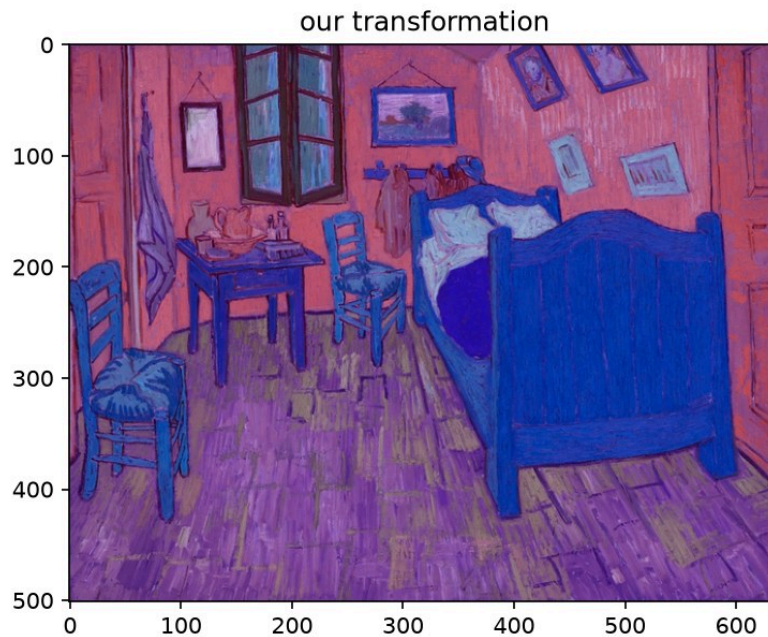
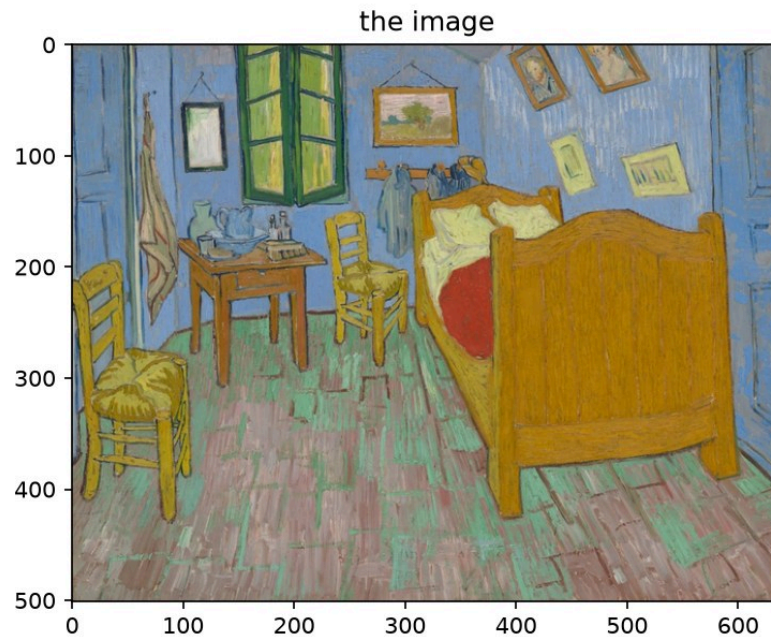
```
darker = grey ** 2  
lighter = np.sqrt(grey)  
negative = 1 - grey
```



A **pointwise** operation applies the same function to each entry separately. The picture keeps its shape; every value changes by the same rule.

Your own transformation

```
def our_transform(image):  
    new = image.copy()           # a copy, so that image stays as it was  
    new[:, :, 0] = image[:, :, 2] # red and blue exchanged  
    new[:, :, 2] = image[:, :, 0]  
    new[:, :, 1] = image[:, :, 1] ** 2  
    return new
```

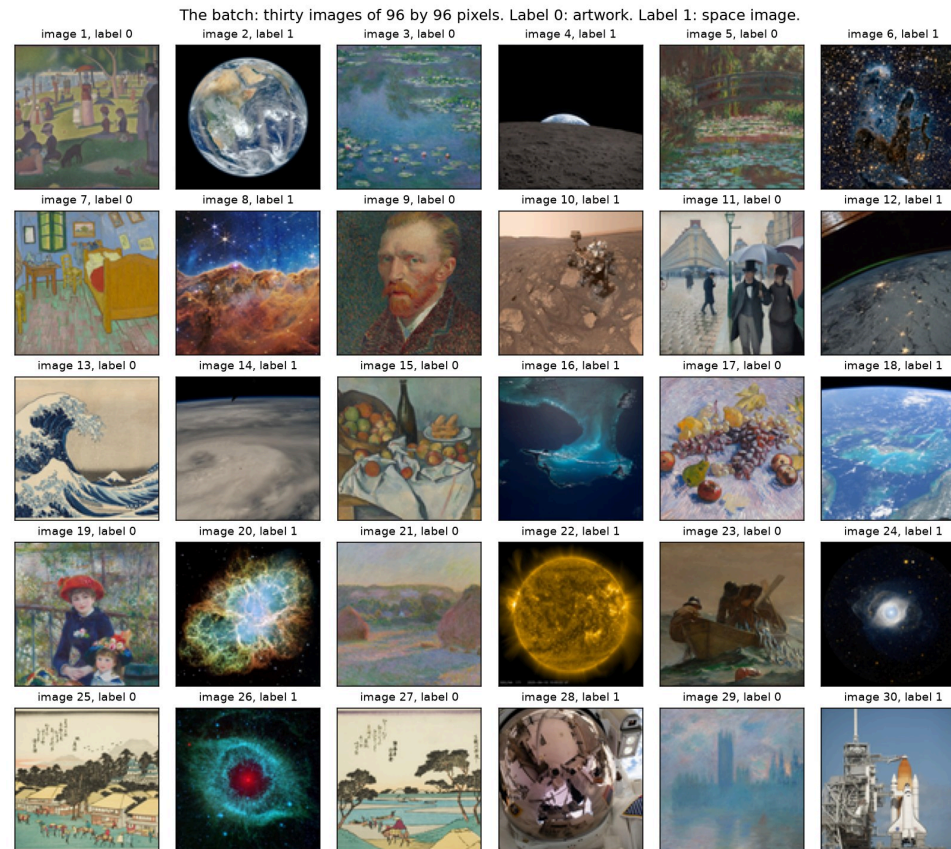


One image is one point

image has shape (501, 640, 3). As one point, it has 961,920 coordinates.
The label of image 7 is 0: artwork.

- In Act 1 a point was one row of the table, with m coordinates. An image is one point too: its coordinates are all of its numbers, $H \times W \times 3$ of them.
- A point can carry a **label**, a value that says of which kind the point is. Each of the thirty images has one: 0 for an artwork, 1 for an image taken in space.

A batch of images



shape of batch: (30, 96, 96, 3) - image, row, column, channel
shape of batch[0], the first image: (96, 96, 3)

One operation for the whole batch

```
grey_batch = 0.299 * batch[:, :, :, 0] + 0.587 * batch[:, :, :, 1] + 0.114 * batch[:, :, :, 2]
brightness = grey_batch.mean(axis=(1, 2))          # one number for each image
X_images = batch.reshape(30, -1)                  # one row for each image
```

```
shape of grey_batch: (30, 96, 96)
shape of brightness: (30,)
shape of X_images:   (30, 27648)
```

```
mean brightness of the artworks (label 0):    0.49
```

```
mean brightness of the space images (label 1): 0.30
```

```
The rule 'darker than 0.35 means space image' gives the right label for 23 of the 30 images.
```

One more colon, and the grey rule converts all thirty images at once. One number for each image, its mean brightness, compared with a threshold, gives the right label for 23 of the 30. A rule of this kind, which converts the numbers of a point into a score, is called a **neuron**.

When one operation is not enough

Every new picture of this act came from arithmetic on whole arrays, and the loop over the pixels ran inside NumPy.

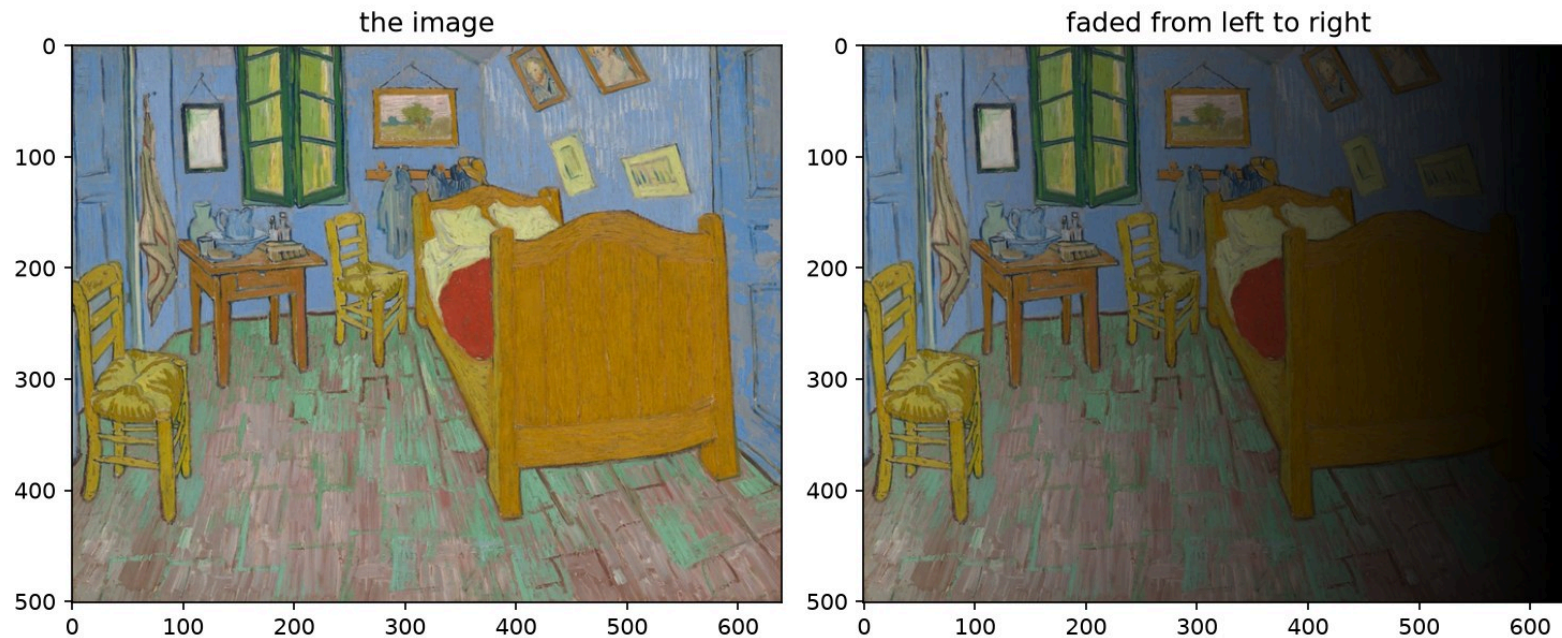
- Think of a transformation of an image that you **cannot** write that way: for example, one in which the new value of a pixel depends on new values already computed for other pixels, so that the pixels must be visited in order.
- When a computation cannot be vectorized, is a slow Python loop the only choice?
- The full notebook of Act 2 asks for your example in Step 18. An assignment of the unit returns to the question.

Now in Colab: Act 2

- **Act 2** of the class notebook: your image, in colour and in grey.
- Complete `our_transform`: a new picture from the channels of your image, by yourselves or with an AI assistant. Exchange channels, set one to zero, apply a pointwise operation to one channel, or something of your own.
- **Post**, as a reply to your group's first post: the picture of your image beside its transformation, with one sentence that says what the transformation does. One member posts; begin with the names of all the members.

A transformation of my own: a fade

```
fade = np.linspace(1.0, 0.0, W)           # one number for each column: 1 at the left, 0 at the right
faded = image * fade[None, :, None]        # broadcast over the rows and the three channels
show([image, faded], ["the image", "faded from left to right"])
```



A function of the pixel's **position** is still one operation: a ramp of W numbers, broadcast over the rows and the channels.